

INM707 Deep Reinforcement Learning Coursework Report

Student Name: Ali Abdulzahra,

Work Mode: Individual Submission.

Github Repository:

Contents

INM707 Deep Reinforcement Learning Coursework Report	1
1. Introduction,	1
2. Basic Q learning in gridworld,	2
3. Baseline Q learning performance,	3
4. Q learning parameter and reward experiments,	4
5. DQN with Double DQN and Dueling DQN,	6
6. Proximal policy optimisation,	7
7. Individual contribution and code submission,	9
8. Conclusion,	10
9. References,	10

1. Introduction,

In this coursework, I implemented and evaluated reinforcement learning methods on two control problems. The first part used tabular Q learning in a Gridworld environment. This was used to define the reinforcement learning problem as a Markov Decision Process, specify the transition and reward functions, train a Q table, and evaluate how different parameters affected learning.

The second part moved from tabular learning to deep reinforcement learning. I implemented DQN on CartPole v1 and compared a baseline DQN against Double DQN, Dueling DQN, and Double Dueling DQN. These variants were used to test whether changes to the value estimation method and network structure improved learning performance.

The final part used Proximal Policy Optimisation on CartPole v1. This gave a comparison against DQN using a policy gradient method. I evaluated PPO through training return, greedy evaluation return, policy entropy, and clip fraction. This allowed me to compare value based learning with an actor critic method.

All experiments were implemented in Python. I used saved training histories, evaluation summaries, and figures to analyse the results. The report follows the coursework marking structure by first covering the basic Q learning tasks, then the DQN improvements, and finally the additional deep reinforcement learning algorithm.

2. Basic Q learning in gridworld,

For the basic reinforcement learning task, I implemented a custom delivery Gridworld. The environment is a 6 by 6 grid where the agent starts at position (5, 0). The agent must first reach the pickup cell at (1, 4) and then reach the delivery cell at (0, 5) while carrying the package. The grid also contains blocked cells at (2, 2), (3, 2), and (4, 4), which cannot be entered. Hazard cells are located at (1, 1) and (3, 4). These cells are legal to enter, but they produce a penalty.

The problem was defined as a finite Markov Decision Process. The state was represented as (row, column, carrying), where carrying is 0 before pickup and 1 after pickup. This made the carrying status part of the state, which was necessary because the same grid position can have a different meaning before and after the package has been collected. After removing blocked cells, the environment had 33 valid grid positions. Since each valid position had two carrying states, the full state space contained 66 states. This follows the standard reinforcement learning view of an agent interacting with an environment through states, actions, rewards, and transitions (Sutton and Barto, 2018).

The action space contained five discrete actions: up, down, left, right, and wait. The transition function was deterministic. Each movement action changed the agent position unless the target cell was outside the grid or blocked. In that case, the agent stayed in the same state and received an invalid movement penalty. The package was picked up automatically when the agent entered the pickup cell. The episode terminated only when the agent reached the delivery cell while carrying the package, or when the maximum episode length of 100 steps was reached.

The reward function combined a shaped learning signal with the final delivery objective. Each valid step received a small cost of -1.0, which encouraged shorter paths. Invalid movement received -5.0, entering a hazard cell received -10.0, first arrival at the pickup cell received 10.0, and successful delivery received 50.0.

Event	Reward
Valid movement or wait	-1.0
Invalid movement into a wall, boundary or blocked cell	-5.0
Entering a hazard cell	-10.0
First arrival at pickup cell	10.0
Successful delivery while carrying the package.	50.0

Table 1. Reward values used in the custom Gridworld environment.

I used tabular Q learning because the environment had a finite state space and a discrete action space. Q learning is suitable here because it learns action values for state and action pairs through temporal difference updates, which matches the discrete representation used in this Gridworld (Watkins and Dayan, 1992; Sutton and Barto, 2018). The Q table stored one value for every state and action pair. The update rule was:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a^1} Q(s^1, a^1) - Q(s, a)]$$

In this equation, α is the learning rate and γ is the discount factor. The baseline learning rate was set to 0.5 and the discount factor was set to 0.95. I trained the agent for 2000 episodes. The exploration policy used epsilon greedy action selection. Epsilon started at 1.0, decayed by a factor of 0.995 after each episode, and was limited to a minimum value of 0.05. This allowed the agent to explore heavily at the start of training and rely more on learned Q values later.

I evaluated the learned policy using greedy action selection with no exploration. This separated training performance from final policy quality, since training episodes still included random exploratory actions. The main evaluation metrics were mean return, mean number of steps, and success rate.

3. Baseline Q learning performance,

The baseline Q learning agent learned the delivery task within the 2000 episode training run. At the start of training, returns were low because the agent explored randomly and often used the full 100 step episode limit. The first evaluation at episode 1 had a mean greedy return of -480.0, a mean episode length of 100.0, and a success rate of 0.0. This shows that the initial Q table did not contain a useful policy.

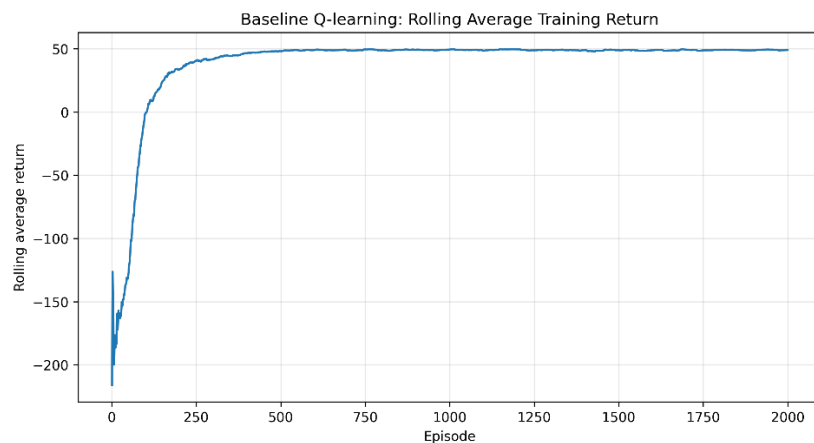


Figure 1. Baseline Q learning rolling training return in the Gridworld environment.

Training improved after early exploration. By episode 50, the greedy evaluation return had reached 50.0, the mean evaluation episode length had fallen to 10.0, and the evaluation success rate had reached 1.0. This means the greedy policy had already found a successful route from the start cell to the pickup cell and then to the delivery cell. The rolling training return then stabilised near 50, which was the maximum return produced by the learned 10 step route under the shaped reward design.

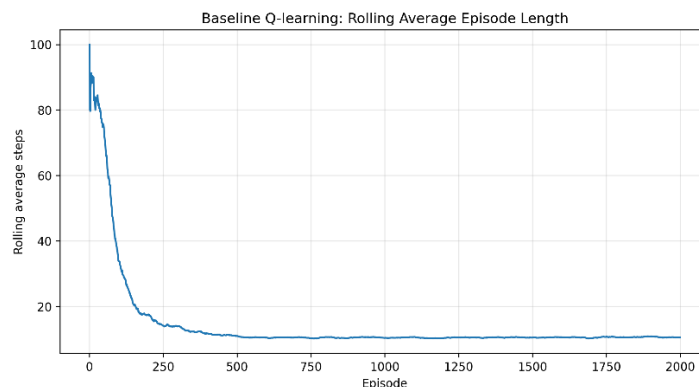


Figure 2. Baseline Q learning rolling episode length in the Gridworld environment.

The rolling episode length also shows that learning was successful. At the start of training, the agent often reached the episode cap because it had not learned how to reach the pickup and delivery cells. As training progressed, the rolling episode length dropped toward about 10 steps.

This matches the greedy rollout saved after training, where the final policy completed the task in 10 actions.

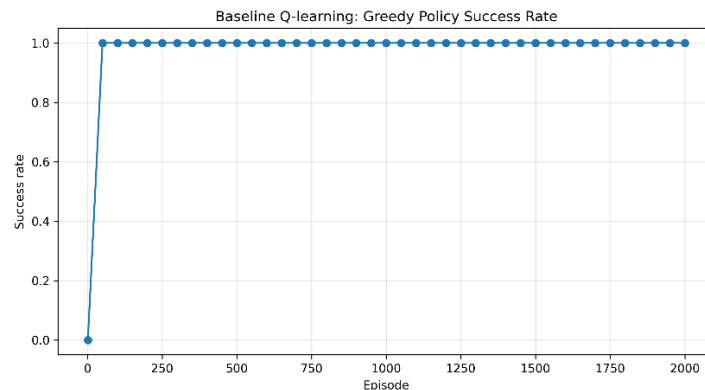


Figure 3. Baseline Q learning greedy policy success rate in the Gridworld environment.

The final greedy rollout followed a direct path through valid cells. The agent moved from (5, 0, 0) to the pickup cell at (1, 4, 1), then moved to the delivery cell at (0, 5, 1). The saved rollout contained 10 transitions. The total return was 50.0, and the final transition ended with successful delivery. This confirmed that the learned policy was not only achieving high training return, but was also solving the task when exploration was removed.

Overall, the baseline Q learning result shows that the tabular method was suitable for this environment. The state space was small enough for a Q table, the deterministic transitions made the optimal route learnable, and the shaped reward helped the agent move from random exploration to a stable policy. The early improvement in greedy evaluation also shows that the agent did not need all 2000 episodes to discover the correct route, although the full run helped confirm that the policy remained stable.

4. Q learning parameter and reward experiments,

I repeated the Q learning experiment with different learning rates, discount factors, exploration schedules, and reward designs. This tested whether the baseline result depended on one narrow parameter setting or whether the task could be solved under a wider set of choices.

The learning rate sweep compared $\alpha = 0.1, 0.5, \text{ and } 0.9$. All three settings reached a similar final rolling return near 50. The main difference was early learning behaviour. A low learning rate updated the Q table more slowly, while larger learning rates reacted faster to new rewards. In this deterministic Gridworld, the higher learning rates did not cause major instability because the transition and reward signals were consistent.

The discount factor sweep compared $\gamma = 0.50, 0.75, 0.95, \text{ and } 0.99$. All settings reached a similar final result. This happened because the successful route was short, so even smaller discount factors could still learn useful action values. Higher discount factors were still more suitable in principle because the task required the agent to value the delayed delivery reward after first collecting the package.

The exploration schedule had a stronger effect on learning speed. The fast decay setting reduced epsilon too early, which made the agent exploit learned values sooner. This produced fast improvement, but it also risked reducing exploration before the state space had been sampled well. The medium decay setting gave a balance between exploration and exploitation

and reached high returns without the long delay seen in the slow decay setting. The slow decay setting kept random action selection for longer, so its rolling return improved more gradually.

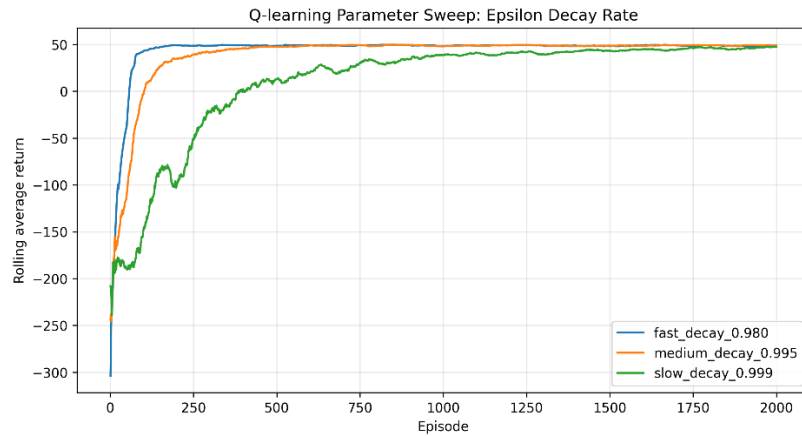


Figure 4. Q learning parameter sweep comparing epsilon decay rates.

I also compared four reward designs: sparse reward, baseline shaped reward, harsh penalties, and weak delivery reward. The sparse reward design performed well once the agent discovered the correct route, but it gave less guidance during early exploration. The baseline shaped reward improved learning by combining a delivery reward with movement costs and pickup reward. The harsh penalty setting slowed early learning because mistakes produced larger negative returns, although the final policy still improved. The weak delivery reward produced the weakest final performance because the main task reward was not large enough compared with the step costs and penalties.

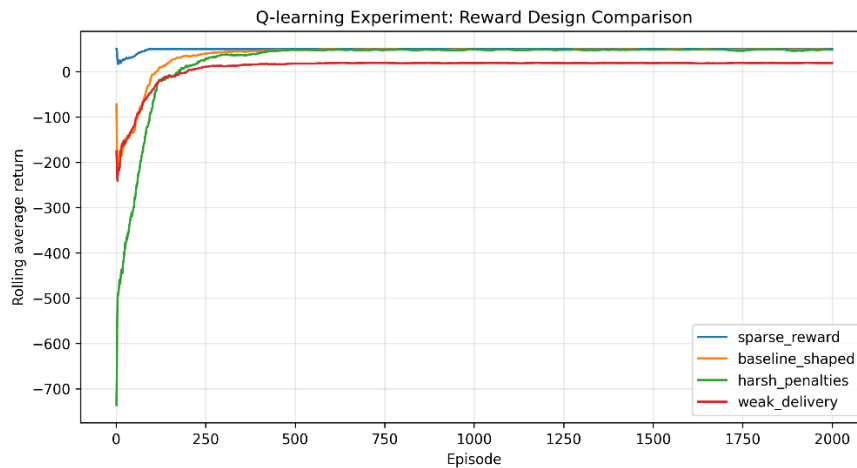


Figure 5. Q learning reward design comparison across four reward structures.

These experiments show that Q learning was robust on this Gridworld, but the exploration schedule and reward design affected sample efficiency. Learning rate and discount factor changed the early curve, but they did not change the final outcome much. In contrast, exploration and reward shaping affected how quickly the agent found and repeated the successful route. This supports the use of shaped rewards in small custom environments, while also showing that the final reward must remain large enough to define the main objective.

5. DQN with Double DQN and Dueling DQN,

For the advanced value based task, I implemented DQN on CartPole v1. CartPole has a continuous state space with four observations: cart position, cart velocity, pole angle, and pole angular velocity. The action space is discrete, with one action pushing the cart left and one action pushing it right. A tabular Q table is not suitable for this state space, so I used a neural network to approximate the action value function.

The baseline DQN used an experience replay buffer, epsilon greedy exploration, a policy network, and a target network. Experience replay reduced correlation between consecutive samples by training on random minibatches from stored transitions. The target network made training more stable by giving a delayed target for the Bellman update. These stabilisation methods follow the DQN approach introduced by Mnih et al. (2015). The agent was evaluated using greedy action selection, so the evaluation return measured the learned policy without random exploration.

I then tested two DQN improvements. The first improvement was Double DQN. In standard DQN, the same target network is used to select and evaluate the next action, which can overestimate action values. Double DQN reduces this by using the policy network to select the next action and the target network to evaluate that action (van Hasselt, Guez and Silver, 2016). The second improvement was Dueling DQN. This changes the network architecture so that it estimates a state value and an action advantage, then combines them into Q values. This can help when many states have similar action values (Wang et al., 2016).

I compared four variants: baseline DQN, Double DQN, Dueling DQN, and Double Dueling DQN. All four variants were able to solve CartPole at some point in training, but their stability differed.

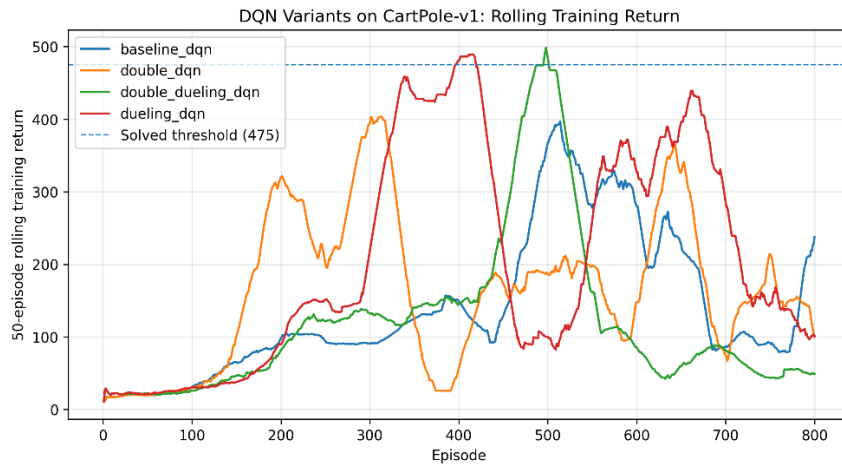


Figure 6. Rolling training return for baseline DQN, Double DQN, Dueling DQN, and Double Dueling DQN.

The rolling training return shows that the dueling variants learned strong policies, but not always in a stable way. Dueling DQN reached the solved threshold by rolling training return at episode 397. Double Dueling DQN reached it later at episode 496, but achieved the highest best rolling training return of 498.48. This showed that the combined architecture could produce a strong training policy, but it did not guarantee the best final evaluation behaviour.

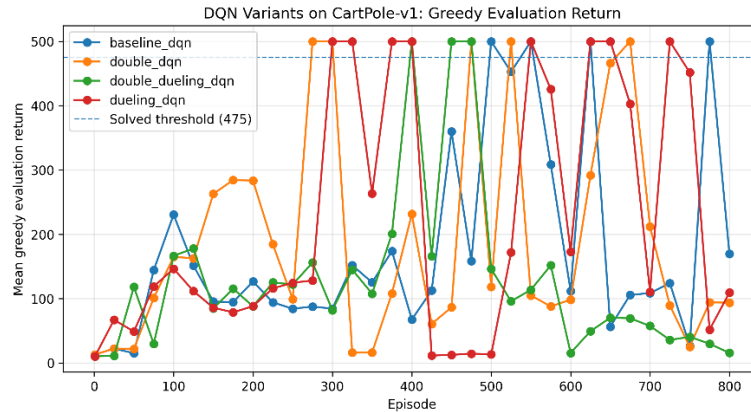


Figure 7. Greedy evaluation return for baseline DQN, Double DQN, Dueling DQN, and Double Dueling DQN.

The greedy evaluation results show the same stability issue. Baseline DQN reached a best evaluation return of 500, but ended with a final evaluation return of 169.8. Double DQN also reached 500 and did so earlier, at episode 275, but its final evaluation return dropped to 93.7. Dueling DQN reached 500 at episode 300 and gave the clearest improvement in learning speed. Double Dueling DQN reached high training returns but ended with a final evaluation return of only 15.5.

Variant	Best evaluation return	Episode of best evaluation	Final evaluation return	Best rolling training return
Baseline DQN	500.0	400	169.8	430.22
Double DQN	500.0	275	93.7	432.60
Dueling DQN	500.0	300	500.0	482.10
Double Dueling DQN	500.0	500	15.5	498.48

Table 2. Summary of DQN variant performance on CartPole v1.

The main result was that DQN could solve CartPole, but performance depended on the variant and on late training stability. Double DQN improved the value update method and reached the maximum evaluation score earlier than the baseline. Dueling DQN gave the strongest final evaluation result because it ended with a greedy evaluation return of 500.0. Double Dueling DQN had the best peak rolling training return, but its final greedy evaluation score collapsed. This suggests that combining improvements does not always produce the best policy unless the training schedule, exploration decay, and target updates are tuned together.

Overall, Dueling DQN gave the best tradeoff in this experiment. It learned faster than the baseline, reached the maximum evaluation return, and retained the strongest final greedy policy.

6. Proximal policy optimisation,

For the second deep reinforcement learning algorithm, I implemented Proximal Policy Optimisation on CartPole v1. PPO is an actor critic method, so it learns both a policy and a value function. The actor selects actions from a probability distribution, while the critic estimates the value of the current state. This makes PPO different from DQN, which learns action values and then chooses the action with the highest estimated value. PPO was chosen because it is designed to improve policy gradient training by limiting the size of policy updates through a clipped objective (Schulman et al., 2017).

The PPO agent collected batches of on policy experience and then updated the actor critic network using the clipped PPO objective. The clipped objective limits how far the new policy can move away from the old policy during one update. This is important because large policy updates can damage a working policy. I also tracked policy entropy and clip fraction. Entropy measured how random the policy remained during training, while clip fraction measured how often the PPO ratio was clipped during updates.

The PPO training return improved over time, but it stayed more variable than the final greedy evaluations. Early in training, the return was low because the policy still selected actions with high randomness. As training continued, returns increased and the agent became more capable of balancing the pole. However, the rolling training return did not stay above the CartPole v1 solved threshold of 475, which is the threshold used in the Gym CartPole v1 environment specification (OpenAI Gym, 2024).

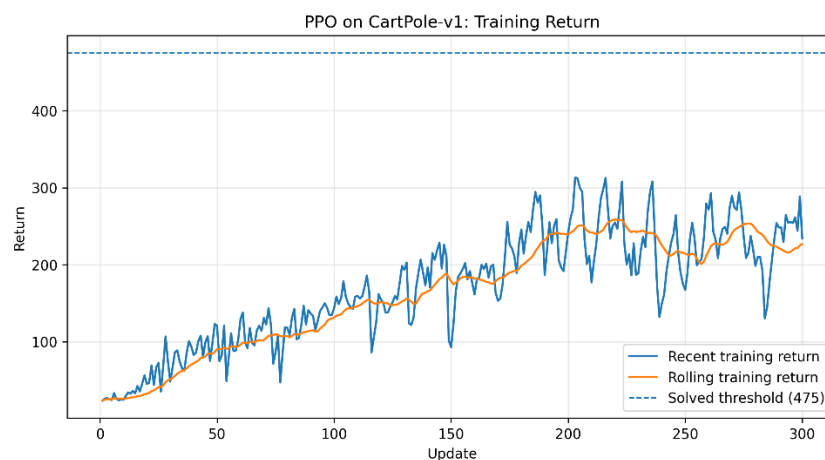


Figure 8. PPO training return on CartPole v1.

The greedy evaluation return showed stronger policy quality than the training return. PPO reached the maximum evaluation return of 500 several times, especially from around update 190 onward. This means the learned policy could solve CartPole when evaluated without sampling noise. The difference between training return and greedy evaluation return happened because training used stochastic action sampling, while greedy evaluation used the most likely action.

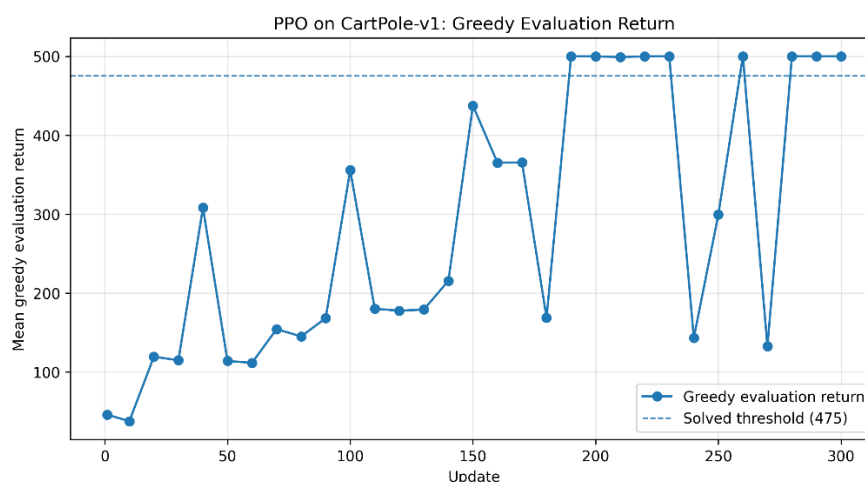


Figure 9. PPO greedy evaluation return on CartPole v1.

Metric	PPO result
Best greedy evaluation return	500.0
Final Greedy evaluation return	500.0
Solved in greedy evaluation	Yes
Training return reached solved threshold	No
Main observed issue	Variable on policy training return

Table 3. Summary of PPO performance on CartPole v1.

The entropy curve decreased during training, which showed that the policy became less random as the agent learned useful actions. This was expected because the policy should explore more at the start and become more decisive once it has learned how to balance the pole. The clip fraction stayed mostly low, with some spikes. This suggests that most updates stayed within the clipped objective, although some batches produced larger policy changes.

Compared with DQN, PPO produced a strong final greedy policy, but the training curve was less direct. DQN variants sometimes reached high returns quickly, but several of them had unstable final evaluation performance. PPO also showed variation during training, but its final greedy evaluation return was 500.0. This made PPO the strongest final policy in the experiments, although not the most stable method during stochastic training.

Overall, PPO was a suitable second algorithm because it solved the same CartPole task through a different learning approach. The results show that both value based methods and actor critic methods can solve CartPole, but they fail in different ways. DQN was sensitive to value estimation and late training collapse, while PPO was sensitive to stochastic policy updates and variable training return.

7. Individual contribution and code submission,

I completed this coursework individually. There were no team members, so all tasks in the report, notebooks, experiments, result generation, evaluation, and analysis were completed by me.

For the basic task, I designed and implemented the custom Gridworld environment, including the state representation, action space, transition function, reward function, Q table, epsilon greedy policy, training loop, greedy evaluation, parameter sweeps, and reward design comparison.

For the advanced value based task, I implemented DQN on CartPole v1. I then extended the baseline DQN into Double DQN, Dueling DQN, and Double Dueling DQN. I trained each variant, saved the model weights, compared training and evaluation results, and analysed the effect of each improvement.

For the second advanced algorithm, I implemented PPO on CartPole v1 using an actor critic network. I trained the policy using the clipped PPO objective and evaluated the learned policy using greedy evaluation. I also saved and analysed training return, evaluation return, entropy, and clip fraction.

The code submission includes the three Jupyter notebooks and the generated result files. The notebooks are written in Python and are structured so that the experiments can be rerun. The saved result files include training histories, evaluation summaries, policy or value network weights, and figures used in this report.

8. Conclusion,

This coursework showed how reinforcement learning methods can be applied at different levels of complexity. In the basic task, tabular Q learning was suitable because the Gridworld had a small finite state space and a discrete action space. The agent learned a successful delivery policy, reached a greedy success rate of 1.0, and completed the final greedy rollout in 10 steps. The parameter experiments showed that learning rate and discount factor affected the early learning curve, but did not change the final result much. Exploration decay and reward design had a larger effect on how quickly the agent found the successful route.

The DQN experiments showed why deep reinforcement learning is needed when the state space is continuous. DQN was able to solve CartPole, and the improved variants also reached the maximum evaluation return of 500 at some point during training. However, the final results showed that high peak performance did not always mean stable learning. Dueling DQN gave the strongest final result because it reached and retained a final greedy evaluation return of 500.0. Double Dueling DQN reached the highest peak rolling training return, but its final evaluation return collapsed, which showed that combining improvements does not always guarantee a better final policy.

The PPO experiment solved CartPole using an actor critic approach rather than a value based approach. PPO reached a final greedy evaluation return of 500.0, even though its stochastic training return remained variable. This showed that the learned policy was effective during evaluation, but that on policy training returns can remain noisy because actions are sampled during training.

Overall, the experiments met the main objectives of the coursework. I represented a reinforcement learning problem as a Markov Decision Process, implemented and evaluated Q learning, tested parameter and reward variations, implemented DQN with two improvements, and applied PPO as a second deep reinforcement learning algorithm. Across the experiments, the main pattern was that successful reinforcement learning depends not only on the algorithm, but also on state representation, reward design, exploration, evaluation method, and training stability.

9. References,

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S. and Hassabis, D. (2015). Human level control through deep reinforcement learning. *Nature*, 518, pp. 529 to 533.

OpenAI Gym. (2024). CartPole v1 environment documentation. Available at the Gym classic control documentation.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.

Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press.

van Hasselt, H., Guez, A. and Silver, D. (2016). Deep Reinforcement Learning with Double Q learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30, pp. 2094 to 2100.

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M. and de Freitas, N. (2016). Dueling Network Architectures for Deep Reinforcement Learning. *Proceedings of the 33rd International Conference on Machine Learning*.

Watkins, C. J. C. H. and Dayan, P. (1992). Q learning. *Machine Learning*, 8, pp. 279 to 292.